DQSA: Dynamic Quantized Self-Attention for Multi-Task Encrypted Network Traffic Classification

Yongming Chen, *Graduate Student Member, IEEE*, Hongsheng Lan, Bah Hwee Gwee, *Senior Member, IEEE*, Qi Cao, Sirajudeen Gulam Razul, and Zhiping Lin, *Life Senior Member, IEEE*

***Abstract*—Network traffic classification is crucial for both network security and management. Despite advances in deep learning-based multi-task traffic classification, existing models often struggle to jointly handle multiple tasks while providing interpretable insights. In multi-task scenarios, different tasks rely on distinct regions of the traffic sequence, motivating the use of dynamic and interpretable attention mechanisms. To this end, we propose Dynamic Quantized Self-Attention (DQSA), a unified framework specifically designed for multi-task network traffic classification. At its core, the Task Gated Attention Router (TGAR) dynamically associates attention heads with different tasks, enabling adaptive focus on task-specific patterns. This mechanism provides interpretable attention scores, which help analyze misclassifications and guide further model refinement. To improve efficiency and handle diverse network traffic features, we introduce the Soft Quantized Self-Attention Head (SQ-SAH) to reduce computational complexity and extend the Rotary Position Embedding (RoPE) to accommodate these features. Extensive experiments on `ISCX VPN-nonVPN` and DCI-LTE datasets demonstrate that DQSA consistently outperforms state-of-the-art baselines, achieving 92.85% accuracy on the encapsulation-level task of `ISCX VPN-nonVPN` and 93.17% accuracy on the application-level task of DCI-LTE, surpassing the strongest existing methods by up to 2.65%, while providing interpretable task-specific attention for efficient multi-task network traffic classification.**

***Index Terms*—Traffic classification, self-attention mechanism, multi-task learning, interpretability.**

I. Introduction

TRAFFIC classification is critical for safeguarding network security and facilitating effective network management [1]. In recent years, deep learning has significantly advanced the traffic classification field. By conducting a deep analysis of network traffic, such as Transmission Control Protocol (TCP) packet flows, various aspects of network traffic can be accurately recognized and classified. This includes the identification of malware [2], Virtual Private Network (VPN) traffic [3], and various network services and applications [4]. Naturally, this scenario lends itself to multi-task learning (MTL), where a single model can handle multiple related classification tasks simultaneously. Existing approaches, including end-to-end MTL [5] and pre-trained MTL models [6], [7], achieve promising performance. However, they often face limitations: different tasks may interfere with each other, and model decisions remain largely opaque, reducing interpretability in practical applications. These challenges motivate the design of a model that can dynamically capture task-specific features while providing inherently interpretable insights.

The effectiveness of deep learning in traffic classification largely stems from its ability to automatically extract complex features from raw data. However, as models grow in depth and complexity, their decision-making processes become increasingly opaque, making it difficult to identify class-discriminative patterns and undermining transparency and trustworthiness. The lack of interpretability limits their real-world applications. Moreover, understanding the decision-making process of a model helps identify weaknesses or biases, enabling targeted improvements. To mitigate this, various post-hoc interpretability methods have been developed and applied in the field of network traffic classification [8], [9], [10]. However, these methods may lead to increased computational overhead and potentially inaccurate explanations. This highlights the need for models that integrate interpretability into the classification process itself, eliminating dependence on post-hoc analysis and reducing the associated risks [11].

While inherent interpretability often involves a trade-off between classification performance and model transparency [12], traditional models such as decision trees or linear models provide transparent decision processes, but they often sacrifice accuracy due to their simplicity. In contrast, deep neural networks capture complex traffic patterns but remain largely opaque. Recently, the Multi-Head Self-Attention (MHSA) mechanism has gained attention for traffic classification [13], [14], since its attention weights naturally offer interpretable insights into which parts of the sequence contribute most

Received 10 November 2024; revised 9 October 2025 and 30 November 2025; accepted 6 December 2025. Date of publication 22 December 2025; date of current version 29 December 2025. The associate editor coordinating the review of this article and approving it for publication was Prof. Haijun Zhang. *(Corresponding author: Zhiping Lin.)*

Yongming Chen, Hongsheng Lan, Bah Hwee Gwee, and Zhiping Lin are with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore 639798 (e-mail: yongming001@e.ntu.edu.sg; lanh0002@e.ntu.edu.sg; ebhgwee@ntu.edu.sg; ezplin@ntu.edu.sg).

Qi Cao is with the School of Computing Science, University of Glasgow, G12 8QQ Glasgow, U.K., and also with the Department of Statistics and Data Science, National University of Singapore, Singapore 117576 (e-mail: qi.cao@glasgow.ac.uk).

Sirajudeen Gulam Razul is with Temasek Laboratories, Nanyang Technological University, Singapore 639798 (e-mail: esirajudeen@ntu.edu.sg).

Digital Object Identifier 10.1109/TIFS.2025.3647211

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.

to predictions [15]. However, directly applying MHSA to multi-task traffic classification presents several challenges. First, incorporating fine-grained traffic features into attention analysis improves fidelity but increases the risk of overfitting due to irrelevant details [16]. Second, the quadratic complexity of MHSA with respect to sequence length raises computational and deployment costs [13]. Finally, and most importantly, existing MHSA-based approaches do not explicitly account for task-specific differences, even though different tasks may depend on distinct regions of the traffic sequence. This motivates the need for a dynamic mechanism that can flexibly associate tasks with attention heads.

To address these challenges, we propose Dynamic Quantized Self-Attention (DQSA), a unified framework for multi-task traffic classification. The central component is the Task Gated Attention Router (TGAR), which dynamically associates attention heads with different tasks. This mechanism explicitly captures both shared and task-specific patterns, improves interpretability, and enables targeted analysis of misclassifications. To enhance efficiency, inspired by [17], we design the Soft Quantization Self-Attention Head (SQ-SAH), which employs a codebook and soft quantization mechanism to constrain the hypothesis space, thereby achieving faster operation, reduced memory usage, and a lower risk of overfitting [18]. Finally, to better handle diverse network metrics, we treat all inputs as time series and enhance the Rotary Position Embedding (RoPE) [19] with a monotonic and multi-velocity variant (M2RoPE), which improves relative position modeling and adapts to traffic characteristics across different protocol layers. Together, these modules make DQSA both efficient and interpretable, while remaining well-suited for real-world multi-task traffic classification.

Our main contributions can be summarized as follows:

- We propose DQSA, a novel framework for multi-task traffic classification that achieves strong performance while offering inherent interpretability.
- We develop the Task Gated Attention Router (TGAR), which dynamically allocates attention heads to tasks, explicitly capturing both task-specific and shared patterns while enhancing interpretability through attention scores.
- We design the Soft Quantized Self-Attention Head (SQ-SAH), which leverages a codebook and soft quantization to constrain the hypothesis space, thereby reducing the quadratic complexity of MHSA, lowering computational and memory costs, and mitigating overfitting risks.
- We extend the Rotary Position Embedding (RoPE) into a monotonic and multi-velocity variant (M2RoPE), improving relative positional modeling and better handling diverse traffic features.
- We evaluate DQSA on the ISCX VPN-nonVPN dataset [20] and a self-collected dataset of Transport Block Size (TBS) in Downlink Control Information (DCI) from cellular networks. Comparisons with state-of-the-art baselines and the post-hoc method DEEPSHAP [21] demonstrate DQSA's superior accuracy, interpretability, and generalizability across diverse network data, with ablation studies confirming the effectiveness of each module.

The remainder of the paper is organized as follows: Section II reviews related work on traffic classification and multiple instance learning. Section III introduces our DQSA method. Section IV details the experiments and results. Finally, Section V provides the conclusion.

II. Related Work

In the early years, traffic classification was typically accomplished using port-based methods or Deep Packet Inspection (DPI) [1], [22]. However, as networks evolved, these techniques have gradually lost their effectiveness. Recently, deep learning-based approaches have rapidly advanced, providing strong performance and ease of implementation. In [23], the feasibility of protocol classification was investigated by analyzing transport-layer payloads using artificial neural networks (ANN) and Stacked Auto-Encoders (SAE). The first 784 bytes of the payload were utilized and converted into a 28×28 image. Then CNN was employed to recognize malware traffic [24] and encrypted traffic [3], respectively. In addition to analyzing payloads, Lopez-Martin et al. extracted both spatial and temporal features from network traffic using CNN and Long Short-Term Memory (LSTM) networks. They focused on the first 20 packets in the network traffic, extracting six key features: source port, destination port, the number of bytes in the packet payload, TCP window size, inter-arrival time, and packet direction [25].

Recently, some studies have leveraged both perspectives of the network traffic mentioned above to enhance model performance. In [26], both payload information and packet-level information were simultaneously utilized. In contrast to the packet information used in [25], the port information was excluded, arguing that the inclusion of port information could lead to performance inflation [27]. Another study [14] explored an alternative approach for extracting temporal information from network traffic. By utilizing both the payload of individual packets and the payload of the entire flow, this method achieves notable performance. In [13], MHSA and LSTM are utilized to extract features from the payload bytes and packet flow, demonstrating strong performance in classifying Transport Layer Security (TLS) protocol traffic.

Building on these advances, another line of work focuses on leveraging large-scale pre-training to further improve traffic classification. Instead of training models from scratch on task-specific datasets, these approaches first pre-train a model on massive packet or flow level traffic datasets and then fine-tune it for specific tasks. For example, in [28], the authors propose a pre-trained datagram-level traffic model for encrypted traffic. Masked autoencoders (MAE) have also been employed to pre-train on large-scale packet and flow level datasets [29], or on masked patch embeddings of traffic bursts [30], achieving accurate traffic classification while significantly reducing the need for labeled data.

Due to the diverse and varied requirements of classification tasks, such as malware detection, service recognition, and traffic classification [22], multi-task learning has attracted attention in the field of network traffic classification. Huang et al. utilized payload information along with a 2D-CNN approach to address multiple tasks simultaneously,

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.

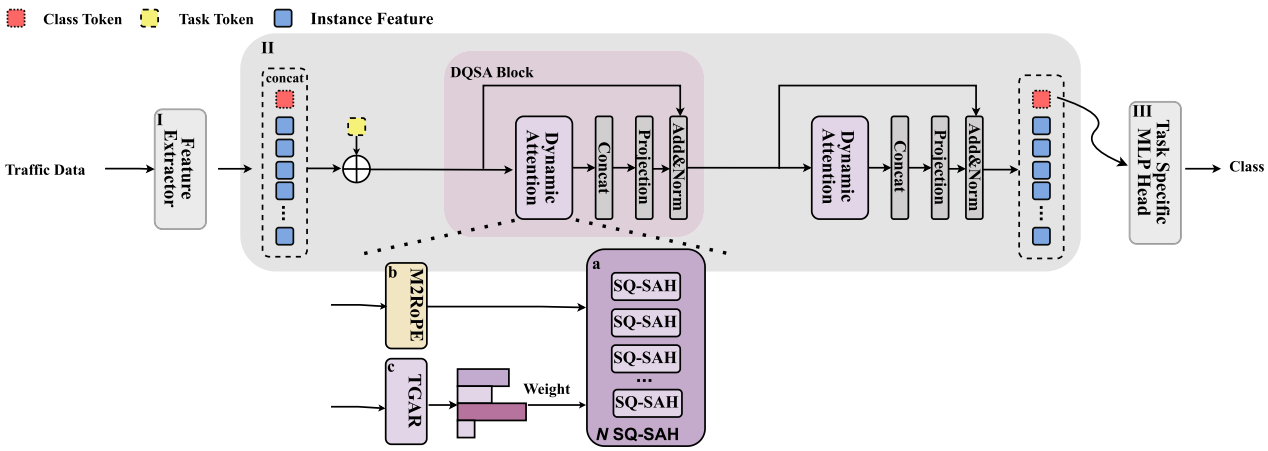


Fig. 1. The framework of DQSA for multi-task traffic classification. (I) The feature extractor captures instance-level feature embeddings. (II) DQSA aggregates instance features into sample-level embedding. Each DQSA block includes a) N SQ-SAH for attention measurement, b) M2RoPE for positional embedding, and c) TGAR for dynamically routing task-specific attention heads. (III) The task-specific classifier maps the sample-level embedding to the corresponding label.

including malware detection, VPN encapsulation recognition, and Trojan classification [31]. Zhao et al. introduced another multi-task deep learning architecture within the framework of federated learning, driven primarily by privacy considerations. In this distributed learning scenario, the tasks addressed include anomaly detection, VPN recognition, and service classification [32]. Another study explored multi-task learning by addressing service classification along with predicting the bandwidth requirements and flow duration [33].

In addition to the aforementioned works, there are also studies that address multi-task learning by integrating payload data with informative protocol fields from network packets. In [34], a two-stage learning framework called Distiller was presented to fuse network traffic information and simultaneously tackle VPN detection, service classification, and application classification. Similarly, Dai et al. [5] designed a Global-Local Attention Data Selection model (GLADS), which combined traffic data by preprocessing it into a specially designed data structure. This structure, processed through a CNN and an attention-based model, achieved the same three classification tasks as Distiller.

It is important to note that these models lack interpretability, which is crucial not only for ensuring the trustworthiness of the model but also for enabling improvements after deployment. Even though the post-hoc method can be conducted in traffic classification model [8], which includes methods based on Shapley values [21], gradient-based approaches [35], and Local Interpretable Model-agnostic Explanations (LIME) [36], the insensitivity to the model or the data reduces some gradient-based methods' reliability for attributing data features [37], [38]. Additionally, some post-hoc interpretation methods, like DeepSHAP, involve huge computation consumption [39]. Moreover, the results of different post-hoc interpretability methods demonstrate the inconsistency [40]. To address this issue, in this paper we proposed DQSA as a solution since DQSA offers inherent interpretability.

Another drawback of existing methods is that some model architectures are specifically tailored to certain input data structures, such as GLADS, which limits their broader applicability. This limitation restricts the model's applicability, as transport layer packets cannot be acquired from all types of networks due to encryption mechanisms. However, alternative network metrics, such as TBS in DCI within cellular networks, can be utilized for traffic classification [41]. To address this issue, we treat all types of input as time series and enhance Rotary Position Embedding (RoPE), making our model more versatile. The rationale behind this approach is that both the informative protocol fields from network packets and the TBS in DCI inherently exhibit a time series structure. Furthermore, the payload is frequently processed using 1D-CNN in numerous studies [5], [24], [34]. For cases where multiple types of network traffic features are input simultaneously, such as payload from the transport layer and packet header information, we will continue referring to this as multimodal to maintain consistency with the terminology used in Distiller [34] and GLADS [5].

III. Methodology

In this section, we present the modules of the DQSA framework and their implementation for multi-task traffic classification. The overall architecture of DQSA is illustrated in Fig. 1. Following the feature extraction from network traffic data, each timestamp is treated as an instance and represented as a feature embedding. The DQSA framework comprises two blocks, each incorporating M2RoPE for positional embedding, TGAR for weighting task-specific attention heads, and multiple SQ-SAH. Each block also includes a feed-forward network (FFN), omitted from the figure for simplicity. Finally, the class token of the sample is utilized for addressing specific traffic classification tasks.

A. Problem Definition

Multi-task traffic classification aims to jointly predict multiple task-specific labels for the same network traffic instance. Formally, given an input traffic sample $\boldsymbol{X} \in \mathcal{X}$, the goal

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.

is to predict the corresponding task-specific labels $\boldsymbol{Y} = [y_1, y_2, \ldots, y_\tau] \in \mathcal{Y}$, where τ denotes the number of classification tasks (e.g., application type, encryption status, or traffic service).

1) Traffic Representation: A traffic instance $\boldsymbol{X}$ may consist of multiple modalities:

$$\boldsymbol{X} = \{\mathcal{M}_1, \mathcal{M}_2, \ldots, \mathcal{M}_J\}, \tag{1}$$

where each modality $\mathcal{M}_j$ corresponds to a distinct feature of the traffic, e.g., transport-layer payload or header information. Each modality is a sequence of length t_j:

$$\mathcal{M}_j = [\boldsymbol{x}_1^j, \boldsymbol{x}_2^j, \ldots, \boldsymbol{x}_{t_j}^j]. \tag{2}$$

Thus, the entire traffic sample is represented as a multi-modal sequence.

2) Transport-Layer Traffic: A flow $\mathcal{F}$ is defined as a sequence of n packets that share the same 5-tuple: transport-layer protocol, source address, destination address, source port, and destination port:

$$\mathcal{F} = \{P_1, P_2, \ldots, P_n\}, \tag{3}$$

where each packet P_i contains: (i) packet-level information $\boldsymbol{PI}_i$ (e.g., length, direction), and (ii) raw payload bytes $\boldsymbol{B}_i$ with q bytes in the i-th packet:

$$P_i = \{\boldsymbol{PI}_i, \boldsymbol{B}_i\}. \tag{4}$$

For example, a TCP flow may contain 10 packets. The packet-level information such as length and direction [5], [25], [34] can be represented as $\boldsymbol{PI}$, and the sequence

$$\mathcal{M}_1 = [\boldsymbol{PI}_1, \boldsymbol{PI}_2, \ldots, \boldsymbol{PI}_{10}] \tag{5}$$

can be considered as one modality of the traffic, preserving its time series property. The payload of each packet P_i can be represented as

$$\boldsymbol{B}_i = [\text{byte}_i^1, \ldots, \text{byte}_i^q], \quad \text{where byte}_i^q \in [0, 255]. \tag{6}$$

Here, i denotes the i-th packet and q is the number of bytes in the packet. The payload sequence of the entire flow can then be represented as

$$\mathcal{M}_2 = [\text{byte}_1^1, \text{byte}_1^2, \ldots, \text{byte}_{10}^q]. \tag{7}$$

Thus, the entire traffic flow can be represented as a multi-modal sequence, $\boldsymbol{X} = \{\mathcal{M}_1, \mathcal{M}_2\}$.

3) Physical-Layer Traffic: For traffic metrics collected from the physical layer, such as the TBS in DCI within cellular networks [41], the network traffic can be represented as a TBS flow. In this case, each feature $\boldsymbol{x}$ corresponds to a TBS value. The entire sample $\boldsymbol{X} = \{\mathcal{M}_1\}$,

$$\mathcal{M}_1 = [\text{TBS}_1, \text{TBS}_2, \ldots, \text{TBS}_t]. \tag{8}$$

Each TBS flow represents a sequence of transmitted block sizes in cellular downlink control information (DCI), which can be used as features for traffic classification.

B. Soft Quantization Self Attention Head (SQ-SAH)

After the feature extraction process, each $\boldsymbol{x}_i^j$ in $\boldsymbol{X}$ is transformed into a feature embedding. For simplicity, without introducing additional variables, we continue to use $\boldsymbol{X}$ to denote the network traffic sample, and $\boldsymbol{x}_i^j \in \mathbb{R}^d$ represents a d-dimensional instance feature embedding. We utilize the *class token* to consolidate information across instances [42]. Additionally, we employ the *task token* to introduce task-specific biases, facilitating tailored adjustments for each task [43]. In this way, we obtain $\boldsymbol{X}_{\boldsymbol{task}}$ for the subsequent processing specific to the task, which can be presented as:

$$\boldsymbol{X}_{\boldsymbol{task}} = [cls, \boldsymbol{x_1}, \boldsymbol{x_2}, \ldots, \boldsymbol{x_L}] + [task] \tag{9}$$

where $[cls]$ and $[task]$ represent *class token* and *task token* respectively and L denotes the length of the sequence $\boldsymbol{X}$.

The MHSA mechanism can be presented in Eq. (10) as follows [44]:

$$\text{MultiHead}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{Concat}\,(\text{head}_1, \ldots, \text{head}_h)\,\boldsymbol{W}^O \tag{10}$$

where each attention head is computed as shown in Eq. (11):

$$\text{head}_i = \text{Attention}\left(\boldsymbol{X}_{\boldsymbol{task}}\boldsymbol{W}_i^Q, \boldsymbol{X}_{\boldsymbol{task}}\boldsymbol{W}_i^K, \boldsymbol{X}_{\boldsymbol{task}}\boldsymbol{W}_i^V\right) \tag{11}$$

The scaled dot-product attention is defined as:

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{Softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_{key}}}\right)\boldsymbol{V} \tag{12}$$

where $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ are the query, key, and value matrices, respectively; $\boldsymbol{W}_i^Q$, $\boldsymbol{W}_i^K$, and $\boldsymbol{W}_i^V$ are the learned projection matrices for the i-th head; $\boldsymbol{W}^O$ is the output projection matrix; and d_{key} is the dimensionality of the keys.

However, the Multi-Head Self-Attention (MHSA) mechanism has a complexity of $\mathcal{O}(L^2)$ due to the computation of the attention matrix $\boldsymbol{Q}\boldsymbol{K}^\top$, which grows quadratically with the sequence length ($L + 1$ in this paper). Therefore, a vector-quantized attention matrix calculation method is introduced in [17]. By leveraging a codebook $\boldsymbol{C} = [\boldsymbol{c_1}, \ldots, \boldsymbol{c_S}]$ and the vector-quantization method, the complexity is reduced to $\mathcal{O}(LS)$, which can be represented as:

$$\Phi\left(\frac{\boldsymbol{Q}\hat{\boldsymbol{K}}^\top}{\sqrt{d_{key}}}\right) = \Phi\left(\frac{\boldsymbol{Q}\boldsymbol{C}^\top\boldsymbol{\Delta}}{\sqrt{d_{key}}}\right) = \Phi\left(\frac{\boldsymbol{Q}\boldsymbol{C}^\top}{\sqrt{d_{key}}}\right)\boldsymbol{\Delta} \tag{13}$$

where $\Phi(.)$ denotes Softmax $(.)$ operation, $\boldsymbol{\Delta} \in \mathcal{R}^{S\times(L+1)}$ and $\boldsymbol{\Delta}_{\mathbf{s,l}} = \delta_{s,z_l}$. Here δ_{s,z_l} denotes the Kronecker delta function:

$$\delta_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} \tag{14}$$

and,

$$z_l = \arg\min_z \|\boldsymbol{k_l} - \boldsymbol{c}_z\|^2 \tag{15}$$

In other words, this method utilizes a fixed-size codebook to control the computational complexity. For each $\boldsymbol{k_l}$ in $\boldsymbol{K}$, the closest codeword $\boldsymbol{c}_{z_l}$ is selected to approximate $\boldsymbol{k_l}$. Nevertheless, due to the involvement of the arg min function, the gradient must be adjusted, for instance, through methods like the Straight-Through Estimator (STE) [45]. To circumvent

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.

this complex process, we introduce the SQ-SAH, which can be formulated as:

$$\text{SQ-SAH}(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}) = \Phi\left(\frac{\boldsymbol{Q}\boldsymbol{C}^\top}{\sqrt{d_{key}}}\right)\Phi\left(\frac{\boldsymbol{C}\boldsymbol{K}^\top}{\sqrt{d_{key}}}\right)\boldsymbol{V} \tag{16}$$

where $\Phi(.)$ denotes Softmax$(.)$ operation. The advantages of SQ-SAH are evident from two perspectives. Firstly, by avoiding the direct calculation of $\boldsymbol{Q}\boldsymbol{K}^\top$, the complexity is reduced to $\mathcal{O}(LS)$. In practice, this leads to a significant reduction in computational complexity, as the calculation of the $(L+1)\times(L+1)$ matrix is avoided. Specifically, the following expression demonstrates the reduction in complexity:

$$\begin{aligned}&\left(\Phi\left(\frac{\boldsymbol{Q}\boldsymbol{C}^\top}{\sqrt{d_{key}}}\right)\Phi\left(\frac{\boldsymbol{C}\boldsymbol{K}^\top}{\sqrt{d_{key}}}\right)\right)\boldsymbol{V}\\ &=\Phi\left(\frac{\boldsymbol{Q}\boldsymbol{C}^\top}{\sqrt{d_{key}}}\right)\left(\Phi\left(\frac{\boldsymbol{C}\boldsymbol{K}^\top}{\sqrt{d_{key}}}\right)\boldsymbol{V}\right)\end{aligned} \tag{17}$$

Another advantage of the SQ-SAH is that it can be interpreted as the *Query* measuring attention to the codebook, while the similarity between each codeword and the *Key* is evaluated by the second $\Phi(.)$ function. This approach effectively constrains the hypothesis space of the attention mechanism, thereby reducing the risk of overfitting [18]. Our experiment results in Section IV demonstrate the effectiveness of the SQ-SAH.

C. Positional Embedding

Since the self-attention module does not inherently capture temporal information, positional embeddings are crucial for self-attention-based models. Although sinusoidal positional embeddings are commonly used [44], they tend to lose relative temporal information, which is important in network traffic classification, during the feedforward process [46]. Recently, RoPE has been a method for encoding positional information in self-attention-based models that enhances the capture of relative positional relationships by rotating embeddings in a continuous space [19]. This approach allows for more effective modeling of sequential data by maintaining the relative distances between positions. Let $\boldsymbol{x}_m \in \mathbb{R}^d$ denote the feature vector at the m-th position. The general form of RoPE for $\boldsymbol{x}_m$, can be presented in Eq. (18).

$$RoPE_{\{q,k\}}(\boldsymbol{x}_m, m) = \boldsymbol{R}^d_{\Theta,m}\boldsymbol{W}_{\{q,k\}}\boldsymbol{x}_m \tag{18}$$

where

$$\boldsymbol{R}^d_{\Theta,m} = \begin{bmatrix} M_1 & & & \\ & M_2 & & \\ & & \ddots & \\ & & & M_{d/2} \end{bmatrix} \tag{19}$$

$$M_i = \begin{bmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{bmatrix} \tag{20}$$

is the rotary matrix with pre-defined parameters $\Theta = \{\theta_i = \theta_{base}^{-2(i-1)/d}, i \in [1,2,\cdots,d/2]\}$.

An illustration of RoPE is shown in Fig. 2, where the input $\boldsymbol{x}_m \in \mathbb{R}^d$ is divided into two-dimensional vectors. Each two-dimensional vector undergoes a rotation based on θ_i, which is determined by θ_{base}, the positional information m, and the dimensional index of the corresponding two-dimensional vector.

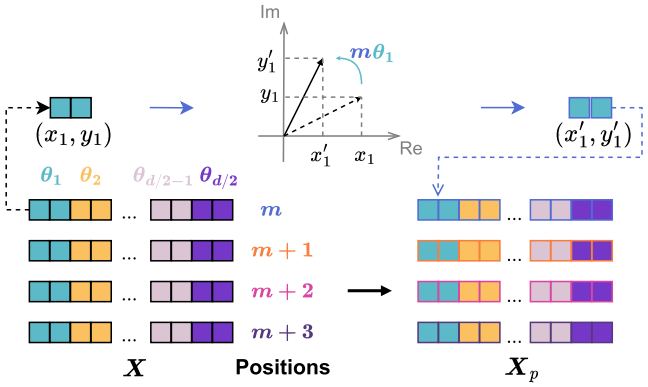


Fig. 2. An illustration of RoPE.

To enhance the effectiveness of RoPE for our task, we made two adjustments to RoPE. First, we scale the position m to $m\sigma$. We observe that interpolating the position improves the monotonicity of the inner product as the relative position increases. To ensure a consistent decay in the magnitude of the inner product with increasing relative position, θ_{base}, which is set as 10000 by default [19], should be reduced to accelerate the rotational transformation. This adjustment ensures that the dot product exhibits the desired monotonic decay as the relative position between vectors grows.

To accommodate the input of network traffic features across multiple modalities, we partition the rotational matrix $\boldsymbol{R}^d_{\Theta,m}$ into two components with different θ_{base}: one that captures the relative positional information within each modality and another that represents the information between different modalities. We hold the assumption that the relative distance within a modality is smaller than the distance between modalities, which is expressed in Eq. (21).

$$d\left(\boldsymbol{x}^i_{t_{i1}}, \boldsymbol{x}^i_{t_{i2}}\right) < d\left(\boldsymbol{x}^i_{t_{i1}}, \boldsymbol{x}^j_{t_{j1}}\right) \tag{21}$$

where $d(.)$ means the relative distance measure and $x^i_{t_{i1}}$ means one timepoint in the i_{th} modality. The distance between two time points within the same modality, $x^i_{t_{i1}}$ and $x^i_{t_{i2}}$, is smaller than that between time points from different modalities, $x^i_{t_{i1}}$ and $x^j_{t_{j1}}$. Because the θ_{base} can be considered as a unit velocity of the rotation, we introduce this difference in the θ_{base} as multi-velocity. Finally, the rotation matrix of the monotonic and multi-velocity variant of RoPE, called M2RoPE, can be expressed as:

$$R = \begin{bmatrix} R^{d_1}_{\Theta_1,m_1\sigma_1} & \\ & R^{d_2}_{\Theta_2,m_2\sigma_2} \end{bmatrix} \tag{22}$$

where d_1 and d_2 are even integers and hold $d_1 + d_2 = d$. In DQSA, we employ M2RoPE in the same manner as RoPE is used in [19].

D. Task Gated Attention Routing (TGAR)

Another module of DQSA is Task-Gated Attention Routing, which enables each attention head to either share functionality

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.

across tasks or be for specific tasks. This flexibility allows the model to leverage commonalities while also focusing on task-specific details, thereby enhancing performance across multiple tasks. To achieve this, the routing network can be represented as:

$$G(\boldsymbol{x})_{task_\tau} = TopK\left(Softmax(\boldsymbol{x}\boldsymbol{W_g}), K\right) \tag{23}$$

where $\boldsymbol{x}$ is from $\boldsymbol{X_{task}}$; $\boldsymbol{W_g} \in \mathbb{R}^{d\times h}$ denotes the router parameters; and $TopK(.,K)$ sets all elements in the vector to zero except for the K largest values. This process allows us to identify the relevant attention heads for the instance $\boldsymbol{x}$ in task $task_\tau$. The final attention can be presented as:

$$\text{DQSA}_{task_\tau} = \text{Concat}\left(G(\boldsymbol{x})_{task_\tau}[\text{head}_1, \ldots, \text{head}_h]\right) \tag{24}$$

However, two issues arise when using the task-gated router directly. First, most instances will utilize only a subset of attention heads or codewords, leading to the other heads or codewords being unused. Second, the task-gated router does not capture the relationships or connections between different tasks. To address these issues, two loss functions are designed in TGAR: Attention Balance Loss and Task Related Loss.

1) Attention Balance Loss: For the model as a whole, our goal is to ensure that each codeword in every attention head is fully utilized to achieve the maximum performance potential of the model. To quantify this, we define the average attention assigned to codeword c_s in attention head h across all tasks and samples as $P(c_s^h)$. The entropy of the attention distribution over all codewords is given by:

$$H(C) = -\sum_{h=1}^{H}\sum_{s=1}^{S} P(c_s^h)\log P(c_s^h) \tag{25}$$

Then, the Attention Balance Loss can be defined as:

$$\text{Loss}_{AB} = \log(H \times S) - H(C) \tag{26}$$

This loss measures the distance between $H(C)$ and its upper bound.

2) Task Related Loss: To illustrate the cooperation and specialization among tasks, we follow the approach outlined in [43]. Specifically, we model $P(H_h|T_\tau)$ as the probability that attention head H_h is used for task T_τ. We then define $P(H_h, T_\tau) = P(H_h|T_\tau)P(T_\tau)$ and $P(H_h) = \sum_T P(H_h, T_\tau)$. To capture the dependence of attention head usage on tasks, we introduce a mutual information measure to quantify this relationship:

$$I(H;T) = \sum_{h=1}^{H}\sum_{\tau=1}^{T} P(H_h, T_\tau)\log\frac{P(H_h, T_\tau)}{P(H_h)P(T_\tau)} \tag{27}$$

Due to the absence of an upper bound for maximizing the Kullback-Leibler (KL) divergence, we choose to use Jensen-Shannon (JS) divergence. We minimize the distance between the JS divergence and its theoretical upper bound to ensure stable training. The JS divergence can be presented as:

$$D_{\text{JS}}(P\|Q) = \frac{1}{2}D_{\text{KL}}(P\|M) + \frac{1}{2}D_{\text{KL}}(Q\|M) \tag{28}$$

where

$$M = \frac{1}{2}(P + Q) \tag{29}$$

Then the Task Related Loss can be defined as:

$$\text{Loss}_{TR} = \log 2 - D_{\text{JS}}(P\|Q) \tag{30}$$

After processing, for a specific task T_τ, we utilize the task-specific MLP head along with the *class token* to compute the task loss shown in Eq. (31).

$$L_\tau = \text{CrossEntropy}(\text{MLP}_\tau([\text{cls}_\tau]), \boldsymbol{Y}_\tau) \tag{31}$$

So the total loss of the multi-task traffic classification is shown in Eq. (32).

$$\text{Loss} = \sum_\tau w_\tau L_\tau + \sum_{\text{layers}}(w_{AB}\text{Loss}_{AB} + w_{TR}\text{Loss}_{TR}) \tag{32}$$

E. Interpretability

The proposed DQSA is naturally interpretable due to its capacity to localize instances of interest within network traffic. This interpretability is achieved by examining the attention maps of the self-attention layers. Specifically, the attention information can be obtained using the *class token*, for the i_{th} SQ-SAH:

$$\boldsymbol{A}_i = \Phi\left(\frac{[cls]\boldsymbol{C}^\top}{\sqrt{d_{key}}}\right)\Phi\left(\frac{\boldsymbol{C}\boldsymbol{K}^\top}{\sqrt{d_{key}}}\right) \tag{33}$$

$$Interpretability = G(X)\mathcal{A} \tag{34}$$

where $\mathcal{A} = [\boldsymbol{A}_i, \cdots, \boldsymbol{A}_h]$ and $\boldsymbol{A}_i \in \mathcal{R}^L$ gives the importance weights of all instances in the network traffic.

F. Complexity Analysis

For a sequence of length L and key dimension d_{key}, the conventional self-attention mechanism requires computing the full attention matrix $QK^\top \in \mathbb{R}^{L\times L}$, which incurs $\mathcal{O}(L^2 d_{key})$ time complexity and $\mathcal{O}(L^2)$ memory consumption. Such quadratic scaling limits the applicability of self-attention to long sequences.

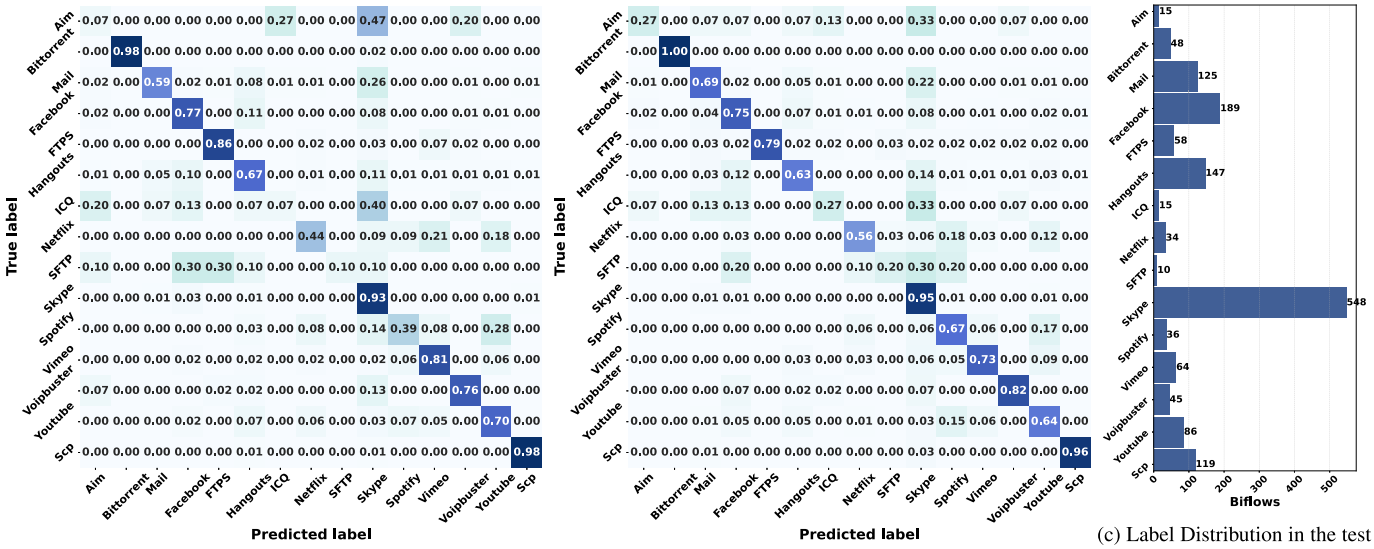
In contrast, the proposed SQ-SAH leverages a fixed-size codebook $C \in \mathbb{R}^{S\times d_{key}}$ to replace pairwise token interactions with token–codeword interactions. As a result, the computation reduces to $\mathcal{O}(LS\,d_{key})$, where S is the codebook size, typically much smaller than L and fixed. This yields linear complexity with respect to sequence length. Moreover, since the $\mathcal{O}(L^2)$ attention matrix is avoided, the memory consumption is reduced to $\mathcal{O}(LS)$.

Therefore, SQ-SAH not only improves computational efficiency but also reduces memory usage, which is particularly beneficial for processing long transport-layer packet sequences. This efficiency improvement is further validated by runtime and GPU memory usage comparisons in Section IV (see Fig. 4).

IV. Experiments and Results

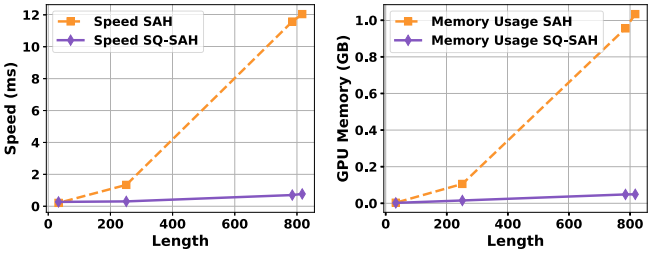
This section begins by introducing the dataset and preprocessing steps. Next, the experimental setup and the baseline models considered are presented. A comprehensive comparison of DQSA against state-of-the-art network traffic classification models across multiple tasks and datasets is subsequently presented, followed by an ablation study to analyze the performance of DQSA. Finally, we demonstrate the performance of DQSA in model pruning and interpretability.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.



(a) Confusion matrix for Distiller (b) Confusion matrix for DQSA (c) Label Distribution in the test set

Fig. 3. Confusion matrix for Distiller and DQSA in the Application Classification task on the ISCX VPN-NonVPN dataset, alongside the label distribution in the test set for this task.



(a) Operation Speed (b) GPU Memory Usage

Fig. 4. Comparison of the speed and GPU memory usage between Self-Attention Head (SAH) and SQ-SAH.

A. Dataset and Preprocessing

To evaluate the performance of our model, we conduct experiments using two datasets shown in TABLE I.

1) ISCX VPN-NonVPN: The ISCX VPN-NonVPN dataset [20] is a well-known public resource for multi-task traffic classification, which provides network traffic data with both standard and VPN-encrypted versions. Each network traffic is labeled with three attributes: encapsulation type, traffic type, and application. For each classification task, there are 2 classes for encapsulation classification, 6 classes for traffic type classification, and 15 classes for application classification.

We implement data cleaning to ensure the quality and relevance of the data. Specifically, we remove biflows that are invalid or do not match the expected labels. This process involves retaining biflows according to the criteria outlined in [47] and excluding those with IP addresses of 0.0.0.0 or 255.255.255.255 [34]. These measures are essential to maintain the integrity of the dataset and ensure that only meaningful and accurate traffic data is used for analysis.

For the input, we use the same two types of input data as in [34]. The first type consists of the first 784 bytes of the transport-layer payload (PAY), which is normalized to a range of [0, 1] [24]. The second type includes header information fields (HDI) from the first 32 packets, incorporating four metrics:

1) Number of bytes in the transport-layer payload
2) TCP window size (or zero for UDP packets)
3) Inter-arrival time
4) Packet direction (0 or 1)

These data types provide a comprehensive feature set for effective classification.

TABLE I

ISCX VPN-NONVPN AND DCI-LTE DATASET TASKS AND CLASSES. CATEGORICAL LABELS ARE REPORTED IN ROUND BRACKETS

Dataset	Task	Classes
ISCX	T_1 - *Encapsulation*	nonVPN (1), VPN (2)
	T_2 - *Traffic Type*	VoIP (1), FileTransfer (2), P2P (3), Streaming (4), Chat (5), Email (6)
	T_3 - *Application*	Skype (1), BitTorrent (2), Hangouts (3), VoipBuster (4), Facebook (5), FTPS (6), SCP (7), Mail (8), YouTube (9), Vimeo (10), Spotify (11), Netflix (12), SFTP (13), Aim (14), ICQ (15)
DCI-LTE	T_1 - *Traffic Type*	StreamingVideo (1), TextChat (2), StreamingMusic (3), VideoChat (4), Game (5), SocialNetworking (6), Shopping (7)
	T_2 - *Application*	YouTube (1), Netflix (2), Instagram (3), X (4), AppleMusic(5), Facebook (6), YouTubeMusic (7), TelegramVideo (8), TelegramText (9), WeChatText (10), WhatsAppText (11), WeChatVideo (12), WhatsAppVideo (13), Carousell (14), Legends (15), PUBG (16), Genshin (17), AmazonPrimeVideo (18), Spotify (19), Lazada (20), Shopee(21)

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.

TABLE II
EXPERIMENTAL CONFIGURATION

Hyperparameter	Value
Number of Attention Heads	8
Number of Codewords	16
Top-K	4
Instance Feature Dimension	128
Dropout Rate	0.2
w_{AB}	0.02
w_{TR}	0.02
w_{τ}	$1/num_{tasks}$
Learning Rate	0.0008
Batch Size	200
Epochs	200

2) DCI-LTE: Another dataset we utilize is called the DCI-LTE dataset, which is self-collected. This experiment addresses the challenge that, in some network frameworks, such as cellular networks, obtaining packet information from the transport layer can be difficult. Our goal is to ensure that our method is applicable to network traffic metrics collected from different network layers. DCI is transmitted unencrypted in the physical layer, including the TBS, reflecting the volume of resources allocated to users in both uplink and downlink. It is hypothesized that variations in traffic utilization across different applications are distinct and that TBS can be employed as a discriminatory feature for traffic classification [41], [48], [49]. The data collection procedure follows the data acquisition system mentioned in [49]. Seven types of network traffic services are collected, with each service represented by traffic from three mobile applications. For each mobile application, 200 TBS flows are collected, with each flow lasting 25 seconds. In total, there are 4200 TBS flows in this dataset.

For preprocessing, we apply a non-overlapping sliding window of 0.1 seconds and compute six features for each window:

1) Maximum TBS for uplink/downlink
2) Total TBS for uplink/downlink
3) Number of DCI instances for uplink/downlink

The class names for each task in each dataset are presented in TABLE I.

B. Setup and Baseline

Here, we introduce the setup of our experiment. The experiments on the two datasets were conducted separately. The dataset was split into three parts: training set, validation set, and test set, with a ratio of 0.7, 0.1, and 0.2, respectively. The model with the highest average accuracy on the multi-task validation set was saved, and its performance was then evaluated on the test dataset. We use InceptionTime [50] as the feature extractor.

The hyperparameters are summarized in Table II. We adopt the Adam optimizer and apply a learning rate scheduler, where the learning rate is reduced by a factor of 0.5 every 45 epochs.

We consider a wide range of existing models for performance comparison.

1) ISCX VPN-NonVPN: The multi-task scenario has been explored in this dataset. To this end, we consider several categories of existing models listed below.

- **Multi-task-Multimodalities** Here, we compare two state-of-art models. Distiller [34] leverages intra- and inter-modality dependencies to utilize the heterogeneity of traffic data and overcome the performance limitations of existing single-modal deep learning-based traffic classification methods. GLADS [5] utilizes an "indicator" mechanism to reconstruct multimodal data, combined with an attention-based framework that captures input features and delivers classification results.
- **Singletask-Multimodalities** We consider the ATVITSC model [14], where payload packet-level images are aggregated into session images. A ViT-like model captures global features, while attention-based convolution extracts temporal features, and a bidirectional LSTM captures spatial features between packets. The fused global and spatiotemporal features are then used to classify encrypted traffic.
- **Multi-task-Singlemodalities** We consider the 2D-CNN model [31], which is a single-modal, multi-task model originally designed for two binary and one multi-class learning task. We have adapted it to fit our requirements, transforming it into one binary learning task (Encaps.) and two multi-class learning tasks (Traffic and App.).
- **Singletask-Singlemodalities** We consider four models in this part, two deep learning models 1D-CNN (PAY) [3], 2D-CNN + LSTM (HDI) [25], and two pretrained models ET-BERT [28] and YaTC [29] as strong baselines. We add task-specific MLP layers to adapt these for a multi-task setting.

It is noted that all models used the same PAY and/or HDI data for a fair performance comparison, except for GLADS and ATVITSC, where special input data constructions are employed. For these cases, we construct the input data as described in their respective papers.

2) DCI-LTE: : Since there are no existing methods specifically designed for multi-task situations in network traffic classification using the TBS in DCI, we consider the following baseline models:

- **Native DL architectures** We consider five deep learning architectures, LSTM, 1D-CNN, 2D-CNN, MLP, and Transformer Encoder, where the first four have been utilized in TBS-based traffic classification [41].
- **Time series classification DL architectures** Because the TBS flow has the characteristics of the time series. We consider three Time series classification DL architectures, i.e., Inceptiontime [50], FCN and Resnet [51]

We implement a classic Transformer baseline (Tr-CLS) for multi-task traffic classification on both ISCX VPN-NonVPN and DCI-LTE datasets. The model uses the same feature embeddings as DQSA, aggregates information via a *[cls]* token, and employs three separate MLP heads for multi-task prediction. This baseline does not include SQ-SAH, TGAR, M2RoPE, or task tokens, providing a fair reference to evaluate the benefits of our dynamic attention and task-gated design.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.

TABLE III

COMPARISON OF DQSA ACCURACY AND F-MEASURE WITH STATE-OF-THE-ART BASELINES ON THE ISCX VPN-NONVPN DATASET. RESULTS ARE PRESENTED AS MEAN ± STD OVER THREE RANDOM REPETITIONS OF EXPERIMENTS. WE HIGHLIGHT THE TOP RESULT WITH BOLD FORMATTING AND THE SECOND-BEST RESULT WITH UNDERLINED FORMATTING

Model	Acc. (Encaps.)	Acc. (Traffic)	Acc. (App.)	F1 (Encaps.)	F1 (Traffic)	F1 (App.)
DQSA(ours)	**92.85 ± 0.66**	**82.49 ± 0.88**	**80.35 ± 0.26**	**92.14 ± 0.67**	**82.05 ± 1.29**	**67.85 ± 0.29**
Distiller [34]	89.55 ± 0.55	81.07 ± 0.81	79.13 ± 0.25	88.01 ± 0.53	80.00 ± 0.88	63.49 ± 1.26
GLADS [5]	90.19 ± 1.08	77.09 ± 1.09	76.02 ± 0.92	87.85 ± 1.12	75.71 ± 0.40	59.18 ± 2.87
ATVITSC [14]	86.61 ± 0.97	75.70 ± 0.80	75.91 ± 1.13	85.41 ± 1.17	73.67 ± 0.70	60.61 ± 1.59
YaTC [29]	91.16 ± 0.79	79.84 ± 2.56	78.64 ± 0.95	90.27± 0.76	80.16 ± 1.99	65.28 ± 1.96
ET-BERT [28]	87.20 ± 0.66	80.15 ± 0.44	79.10 ± 0.84	85.84 ± 0.53	81.05 ± 0.20	65.32 ± 0.32
Tr-CLS	87.52 ± 1.46	78.49 ± 0.34	77.02 ± 1.01	86.54 ± 1.69	78.20 ± 1.33	63.75 ± 1.34
2D-CNN_HUANG(PAY) [31]	80.54 ± 1.74	72.35 ± 0.98	75.46 ± 0.82	77.20 ± 1.82	70.41 ± 1.50	61.57 ± 1.23
1D-CNN_WANG(PAY) [3]	81.58 ± 1.26	73.89 ± 0.47	75.58 ± 1.32	80.76 ± 1.87	73.21 ± 1.32	61.34 ± 0.34
2D-CNN+LSTM(HDR) [25]	87.56 ± 1.37	75.81 ± 1.19	73.67 ± 0.85	84.67 ± 1.85	72.74 ± 1.45	58.57 ± 0.70

TABLE IV

COMPARISON OF DQSA ACCURACY AND F-MEASURE WITH STATE-OF-THE-ART BASELINES ON THE DCI-LTE DATASET. RESULTS ARE PRESENTED AS MEAN ± STD OVER THREE RANDOM REPETITIONS OF EXPERIMENTS. WE HIGHLIGHT THE TOP RESULT WITH BOLD FORMATTING AND THE SECOND-BEST RESULT WITH UNDERLINED FORMATTING

Model	Acc. (Traffic)	Acc. (App.)	F1 (Traffic)	F1 (App.)
DQSA(ours)	**96.19 ± 0.42**	**93.17 ± 0.55**	**96.19 ± 0.41**	**93.19 ± 0.56**
Tr-CLS	95.31 ± 0.27	90.52 ± 1.12	95.32 ± 0.28	90.51 ± 1.19
LSTM	93.29 ± 0.75	82.50 ± 1.41	93.30 ± 0.75	82.59 ± 1.43
1D-CNN	90.28 ± 0.74	81.58 ± 1.27	90.28 ± 0.74	81.66 ± 1.24
2D-CNN	86.03 ± 1.20	68.45 ± 1.51	86.03 ± 1.24	68.49 ± 1.38
MLP	76.07 ± 1.66	51.74 ± 2.06	75.65 ± 1.86	50.23 ± 2.56
INCEPTION [50]	88.65 ± 1.03	73.01 ± 0.43	88.60 ± 1.06	72.78 ± 0.28
FCN [51]	89.88 ± 0.95	72.34 ± 2.11	89.83 ± 0.96	72.08 ± 2.15
RESNET [51]	87.34 ± 0.31	68.37 ± 1.61	87.27 ± 0.34	67.88 ± 1.10

C. Overall Comparison

To evaluate the performance of the DQSA, we use *accuracy*, which represents the proportion of correctly classified samples, and *macro F1-score*, which is calculated as the average of the F1-scores across all classes for each task. The results are shown in TABLE III and TABLE IV.

In our comparative analysis of the DQSA model against state-of-the-art baselines using the ISCX VPN-NonVPN and DCI-LTE datasets, DQSA consistently demonstrates superior performance. On the ISCX VPN-NonVPN dataset shown in TABLE III, DQSA achieves the highest accuracy in all categories, with 92.85% for Encapsulation, 82.49% for Traffic, and 80.35% for Application. DQSA also attains the best F-measure scores: 92.14 for Encapsulation, 82.05 for Traffic, and 67.85 for Application. The next best models, such as Distiller and YaTC, exhibit notably lower performance metrics. Relative to the classical Transformer baseline (Tr-CLS), DQSA achieves accuracy gains of 5.33%, 4.00%, and 3.33% on the three tasks, respectively. Furthermore, when compared with all baseline models, DQSA consistently delivers absolute improvements ranging from 1.69% to 12.31% in encapsulation classification, 1.42% to 6.79% in traffic service classification, and 1.22% to 6.68% in application classification.

Similarly, on the DCI-LTE dataset shown in TABLE IV, DQSA excels with accuracy scores of 96.19% for Traffic and 93.17% for Application. It also exhibits F-measure scores of 96.19 for Traffic and 93.19 for Application. Other models, such as Tr-CLS and INCEPTION, lag behind in these metrics. DQSA achieves accuracy improvements of 0.88% and 2.56% compared with Tr-CLS on the two tasks respectively. Overall, consistent top performance of DQSA is observed in both accuracy and F-measure across diverse network traffic data, which shows its effectiveness in traffic classification tasks.

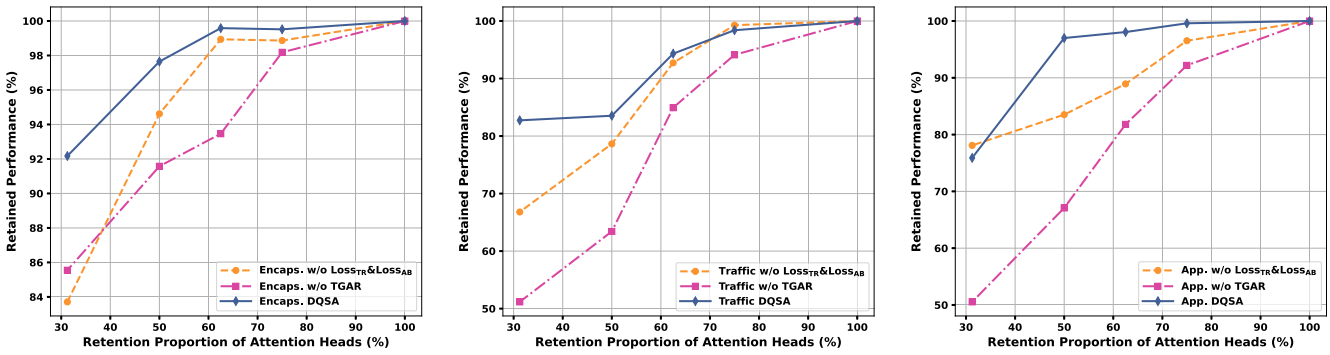
Another advantage of DQSA is evident in its F1 score for the Application Classification task on the ISCX VPN-NonVPN dataset. This dataset is imbalanced, and to ensure a fair comparison, we did not apply any techniques for handling class imbalance, such as oversampling or specialized loss functions, but instead used the same operation as models in [5] and [34]. Under these conditions, DQSA achieves an F1 score of 67.85, surpassing all other models. Compared with the classical Transformer baseline (Tr-CLS), DQSA improves the F1 score by 5.6, 3.85, and 4.1 points across the three tasks. Against all other baselines, DQSA demonstrates consistent absolute F1 score gains, ranging from 1.87 to 14.94 in encapsulation classification, 1.00 to 11.64 in traffic service classification, and 2.53 to 9.28 in application classification. In Figure 3, we visualize the confusion matrix for DQSA and Distiller for this task, as well as the label distribution of the data. It is observed that while Distiller struggles to effectively differentiate categories with few samples, DQSA excels in this aspect.

These consistent improvements, ranging from the classical Transformer baseline to the best-performing state-of-the-art models, demonstrate that DQSA provides not only robustness across tasks and datasets but also tangible advantages in challenging settings such as imbalanced application classification.

D. Ablation Study

In this subsection, we conduct ablation studies on each component of the DQSA model to demonstrate the role and significance of each part in the overall performance of the

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.



(a) **Encaps.** (b) **Traffic** (c) **App.**

Fig. 5. Parameter-priority pruning method. We prune fixed attention heads, aiming to retain as much model performance as possible.

TABLE V
PERFORMANCE COMPARISON BETWEEN SAH AND SQ-SAH

Model	ISCX Acc.	ISCX F1	DCI Acc.	DCI F1
SQ-SAH(ours)	**83.29**	**78.27**	**93.23**	**93.25**
SAH	79.51	73.88	92.61	92.64

TABLE VI
PERFORMANCE COMPARISON BETWEEN DIFFERENT POSITIONAL EMBEDDING METHODS

Model	ISCX Acc.	ISCX F1	DCI Acc.	DCI F1
DQSA(ours)	**85.23**	**80.67**	**94.68**	**94.69**
None	84.60	79.93	94.60	94.61
Sinusoidal	84.47	80.09	94.63	94.62
1DROPE	84.69	79.93	93.39	93.40

model. To ensure clarity, we present the average accuracy and F1 score across all tasks for a given dataset.

1) SQ-SAH: We replace SQ-SAH with different variants of SAH to compare changes in runtime and GPU consumption by fixing the batch size at 50. The results are shown in Fig. 4, where it is evident that as the sequence length increases, both the runtime and GPU consumption for SAH rise dramatically. In contrast, SQ-SAH does not exhibit a significant increase in these metrics. We demonstrate that SQ-SAH can efficiently handle longer sequences, such as extended packets of data retrieved from the transport layer. From the model results shown in Table V, SQ-SAH exhibits enhanced performance, especially on the ISCX dataset. This improvement is attributed to its use of codebooks, which helps mitigate overfitting. As a result, SQ-SAH is able to deliver better outcomes, particularly on imbalanced datasets like ISCX. It is notable that to conduct a fair comparison, we removed the TGAR module but kept *[task]* token in this experiment.

2) Positional Embedding Method: We present the performance of the model using different positional embedding methods. Specifically, we compare sinusoidal embeddings, the original RoPE encoding, and a version without positional embeddings. The results in Table VI demonstrate that our positional encoding yields the best performance on both

TABLE VII
PERFORMANCE COMPARISON BETWEEN WITH OR W/O TGAR

Model	ISCX Acc.	ISCX F1	DCI Acc.	DCI F1
DQSA(ours)	**85.23**	**80.67**	**94.68**	**94.69**
w/o Loss_{TR}	84.46	79.38	93.98	93.99
w/o Loss_{AB}	84.63	80.13	94.40	94.41
w/o Loss_{TR}&Loss_{AB}	84.62	80.30	93.57	93.58
w/o TGAR	83.29	78.27	93.23	93.25

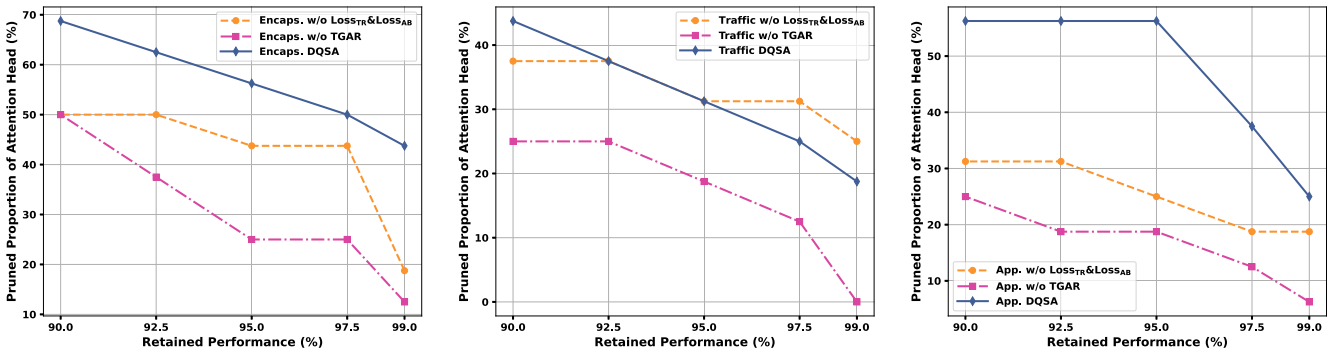
datasets. Given the strong temporal characteristics of the data, ineffective encoding methods can even slightly degrade the model's performance. A comparison with the original RoPE results also shows that slight adjustments to RoPE's θ_{base}, along with the introduction of modality-specific encodings, can enhance the effectiveness of the positional embedding.

3) TGAR: We ablate TGAR and its associated loss functions separately to observe the impact on model performance, with results presented in Table VII. From the results, we can see that incorporating the router significantly enhances the model's performance. Additionally, including the balance loss for codewords and the task-attention relationship loss further improves the model's effectiveness. Our experiments also reveal that these loss functions not only boost performance but also encourage more sparse and focused usage of attention heads, which provides possibilities for model pruning. We will present the results of this pruning experiment in the following section.

E. Model Pruning

In this section, we demonstrate the pruning capability of DQSA. As mentioned earlier, with the inclusion of the TGAR loss function, DQSA exhibits a more sparse codeword usage distribution across different tasks, reflecting both task cooperation and specialization. In this section, all experiments are conducted on ISCX dataset because it is more complex. We here first illustrate the definition we used to measure the variation in the model performance. We define the retained performance after pruning is $Acc_{pruned}/Acc_{original}$. Fig. 7 illustrates the codeword usage distribution in the first layer of DQSA. We normalized the usage probabilities so that the sum

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.



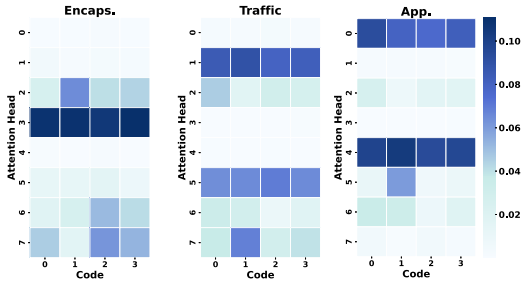
(a) **Encaps.** (b) **Traffic** (c) **App.**

Fig. 6. Performance-priority pruning method. We retain fixed performance, aiming to prune as many attention heads as possible.

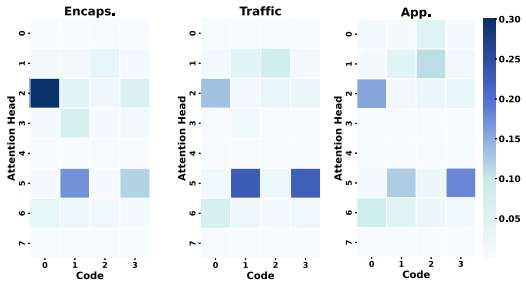
across the eight attention heads and four codewords equals one. The results show that, with the TGAR loss, DQSA effectively captures task cooperation, as evidenced by the sharp utilization of the fifth and second attention heads across multiple tasks. Additionally, task specialization is observed, such as the exclusive use of the third head for Encapsulation classification. To this end, we introduce two prune strategies.

1) **Parameter-priority pruning method**: In this approach, the priority is given to pruning a fixed proportion of attention heads. Specifically, we remove a predetermined percentage of attention heads in the model, with retained model performance serving as the primary metric, where higher values indicate better outcomes. In our experiment, we progressively prune attention heads based on their importance in the variation in the performance, starting from the lowest. The model includes 16 SQ-SAH heads, and we experiment by pruning 4, 6, 8, and 11 heads. The impact of this pruning on model performance is shown in Fig 5.
2) **Performance-priority pruning method**: In this approach, the primary focus is on preserving model performance. Specifically, we define a target proportion of the original performance to retain and use the proportion of attention heads that can be pruned as the metric, where a higher pruned proportion indicates better efficiency. In our experiment, we aim to retain 99%, 97.5%, 95%, 92.5%, and 90% of the model's performance and analyze how many attention heads can be pruned while maintaining these levels. The results are presented in Fig. 6.

For the results of the parameter-priority pruning method, across three tasks, applying a fixed proportion of pruning generally shows that incorporating TGAR preserves higher model performance. Besides, the usage of the $Loss_{TR}$ and $Loss_{AB}$ further enhances the model performance when the fixed amount of the parameters are pruned. For instance, in the Encapsulation task, using TGAR loss allows the model to retain nearly 98% of its performance even after pruning 50% of the attention heads. This is advantageous for practical model deployment, as it suggests that after training a multi-task classification model, modular properties exist between tasks. As a result, fewer parameters can be used to deploy



(a) Codewords usage distribution with TGAR loss function



(b) Codewords usage distribution w/o TGAR loss function

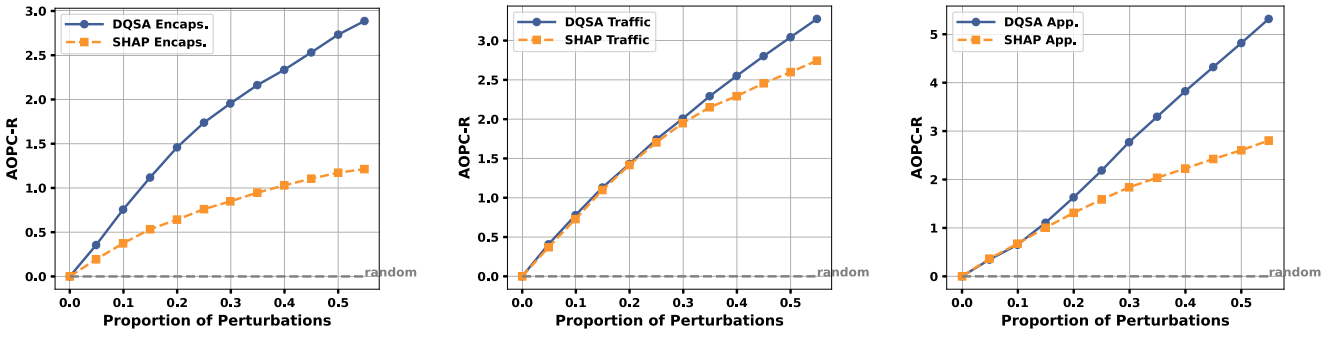
Fig. 7. Codewords usage distribution over the three tasks in ISCX VPN-NonVPN with and w/o TGAR loss function.

a single-task model. Moreover, this characteristic is inherent to the model and does not require complex operations, such as knowledge distillation.

It is noteworthy that for the application task, when pruning and retaining only around 30% of the attention heads, we observe a significant performance drop when TGAR loss is applied, even falling below the performance without the loss. This occurs because, with the addition of TGAR loss, the model allocates more effective attention heads to complex tasks to enhance their performance, as evidenced by the ablation experiments in the previous section. Therefore, we argue that for complex tasks, retaining a larger number of parameters during pruning is essential to maintain performance, as excessive pruning can lead to substantial performance degradation.

A similar conclusion can be drawn from the results of the performance-priority pruning method. For simpler tasks, such as Encapsulation, even when requiring the model to retain 90%

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.



(a) **Encaps.** (b) **Traffic.** (c) **App.**

Fig. 8. AOPC-R in ISCX VPN-NonVPN dataset.

of its original performance, we are still able to prune nearly 70% of the attention heads. However, for more complex tasks, excessive pruning should be avoided to prevent performance degradation.

F. Interpretability Performance

DQSA is inherently an interpretable model by leveraging the self-attention mechanism. In this section, we first demonstrate the effectiveness of DQSA's interpretability by comparing it with the interpretability offered by a well-known post-hoc interpretability analysis method, DEEPSHAP [21]. In this paper, we employ a quantitative comparison of interpretability rather than relying on qualitative visualizations [5], [15]. We believe the quantitative approach provides a clearer understanding of whether the interpretability is indeed effective. In this paper, we use the area over the perturbation curve (AOPC) [52] as the metric to measure the interpretability effectiveness of the DQSA and DEEPSHAP. This idea posits that the effectiveness of interpretability for a sample lies in its ability to highlight what distinguishes one category from another. Therefore, when we progressively remove information from the sample in the order of importance, it should impact the classification outcome accordingly. The *most relevant firstly* (MoRF) removed procedure can be expressed as:

$$\mathcal{X}^{(0)}_{MoRF} = \mathcal{X}$$
$$\forall 1 \le k \le L : \mathcal{X}^{(k)}_{MoRF} = g\left(\mathcal{X}^{(k-1)}_{MoRF}, r_k\right) \tag{35}$$

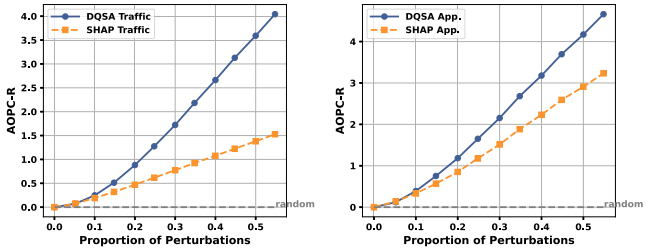
where $g(\cdot)$ denotes the removing operation of the data point that is ranked at position r_k. In this paper, we use the accuracy decay over the test set to measure the impact of the perturbation. So the *AOPC* is expressed in Eq. (36).

$$AOPC = \sum_{i=0}^{L} ACC\left(\mathcal{X}^{(0)}_{MoRF}\right) - ACC\left(\mathcal{X}^{(i)}_{MoRF}\right) \tag{36}$$

where L represents the number of perturbation steps and $ACC(\cdot)$ denotes the accuracy of the corresponding perturbed input text set. Then the definition of the AOPC relative randomly perturbed (AOPC-R) is shown in Eq. (37).

$$AOPC\text{-}R = AOPC_{Interpretability} - AOPC_{Random} \tag{37}$$

where $AOPC_{Random}$ means the *AOPC* with random perturbation.



(a) **Traffic.** (b) **App.**

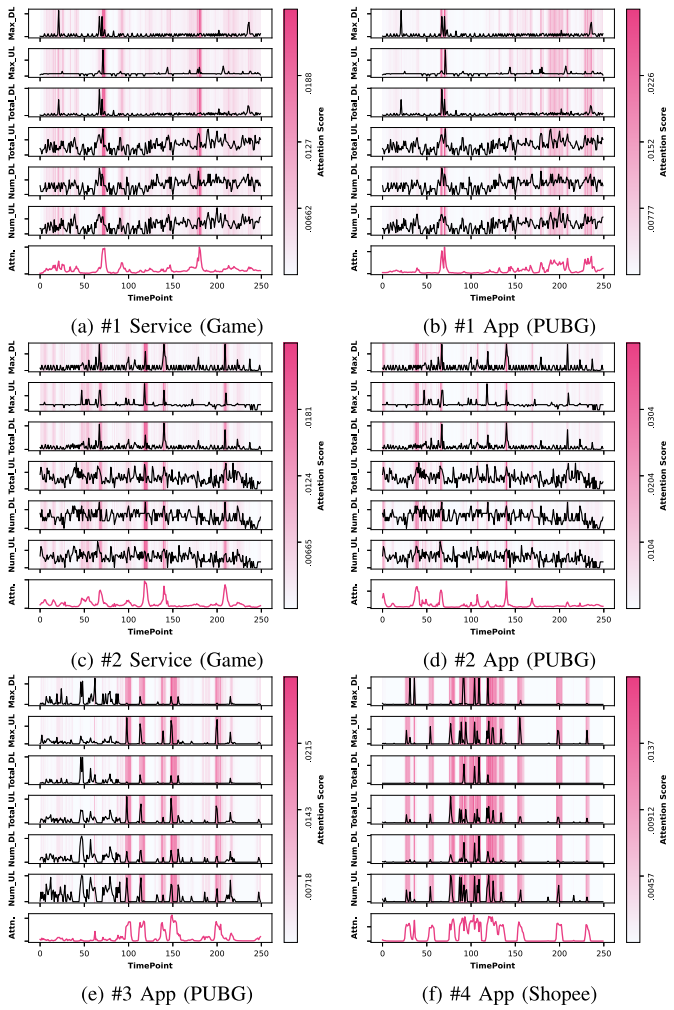
Fig. 9. AOPC-R in DCI-LTE dataset.

In this experiment, we perturb data points ranging from 5% to 50% based on interpretability annotations provided by DQSA and DEEPSHAP. The AOPC-R results are presented in Fig. 8 and Fig. 9.

From the results, we observe that random erasure, serving as the baseline in our experiments, is at zero. In each task, DQSA consistently outperforms DEEPSHAP. This indicates that removing data points based on the interpretability scores provided by DQSA, ordered from highest to lowest, has a significantly greater impact on performance compared to random erasure and also exceeds the impact of DEEPSHAP, a post-hoc analysis method. This demonstrates that DQSA possesses effective intrinsic interpretability, capable of identifying instances that significantly affect model performance. This insight is valuable for understanding the model and for subsequent model enhancement.

We further illustrate the task-specific interpretability of DQSA by visualizing the weighted attention scores derived from Eq. (33) and Eq. (34) for four representative LTE-DCI traffic samples in Fig. 10. Samples #1 and #2 (Game, PUBG) are correctly classified, showing distinct attention patterns across traffic service classification and application classification. Sample #3 (also PUBG) is misclassified as Shopee and its attention map resembles that of sample #4 (Shopee) rather than #1/#2, highlighting how attention visualization can explain misclassification and aid debugging. Specifically, in the "Total_UL" channel, unlike the continuous signals in typical gaming samples that reflect the constant stream of user inputs and synchronization packets required by real-time gaming, Sample #3 exhibits intermittent spikes similar to #4, which misleads the attention mechanism. This analysis can also guide data inspection and model refinement, e.g.,

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.



(a) #1 Service (Game) (b) #1 App (PUBG)

(c) #2 Service (Game) (d) #2 App (PUBG)

(e) #3 App (PUBG) (f) #4 App (Shopee)

Fig. 10. Attention visualization of four samples(#num Task (true label)). #1 and #2 show distinct attention patterns across service and application tasks. #3 is misclassified into the same class as #4 (Shopee), and its attention resembles #4 rather than #1/#2, supporting interpretability and error analysis.

TABLE VIII

PERFORMANCE OF DQSA AND BASELINE MODELS UNDER DISTRIBUTION SHIFTS

Model	Original	DiffBS	LowSig
DQSA	**96.19**	**93.21**	**87.50**
Tr-CLS	95.31	90.00	80.71
LSTM	93.29	87.14	66.79

identifying potential label noise or designing targeted data augmentation to improve robustness.

G. Robustness to New Scenarios

Robustness is a practical concern when deploying traffic classification models in real-world networks. To evaluate this, we collected TBS flows from two different network environments in cellular networks: one from a different base station and another under low signal quality conditions. We tested traffic service classification using three models trained on the original dataset. Table VIII reports the performance of DQSA and two baseline models under these scenarios. Here, "Original" refers to the training dataset, while "DiffBS" and "LowSig" represent distribution shifts relative to the training data.

As shown, DQSA consistently outperforms the baselines across all scenarios. While all models experience some performance drop under new conditions, DQSA demonstrates smaller degradation (e.g., 96.19% → 87.50% in LowSig), indicating stronger generalization and robustness to unseen traffic patterns. These results suggest that DQSA can maintain reliable classification under moderately shifted scenarios. We note, however, that handling severe distribution shifts or entirely new traffic classes remains a challenging and important direction for future research.

V. CONCLUSION

We propose a multi-task network traffic classification model, Dynamic Quantized Self-Attention (DQSA), with inherent interpretability. DQSA incorporates SQ-SAH to reduce complexity and mitigate overfitting, enhanced RoPE encoding for diverse input data, and TGAR for dynamic task-specific attention allocation. We introduce two loss functions for balancing attention load and aligning task-specific focus. Extensive experiments using the ISCX VPN-NonVPN and self-collected DCI-LTE datasets show that DQSA outperforms state-of-the-art models, achieving superior accuracy and F1-scores, especially for classes with fewer samples. Additionally, our model supports task-specific pruning, maintaining 97.5% performance in the Encapsulation classification even after significant pruning. DQSA also demonstrates superior interpretability compared to post-hoc methods, offering valuable insights for future model improvements.

However, our work has limitations. We consider each timestamp as the smallest unit impacting the prediction results. However, in different traffic classification problems, various channels may contribute differently. Future works will address this issue by exploring more sophisticated traffic data representations.

REFERENCES

[1] A. Azab, M. Khasawneh, S. Alrabaee, K.-K.-R. Choo, and M. Sarsour, "Network traffic classification: Techniques, datasets, and challenges," *Digit. Commun. Netw.*, vol. 10, no. 3, pp. 676–692, Jun. 2024.

[2] X. Zhang et al., "Enhanced few-shot malware traffic classification via integrating knowledge transfer with neural architecture search," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5245–5256, 2024.

[3] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Informat. (ISI)*, Jul. 2017, pp. 43–48.

[4] Y. Yang et al., "A network traffic classification method based on dual-mode feature extraction and hybrid neural networks," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 4, pp. 4073–4084, Dec. 2023.

[5] J. Dai, X. Xu, and F. Xiao, "GLADS: A global-local attention data selection model for multimodal multitask encrypted traffic classification of IoT," *Comput. Netw.*, vol. 225, Apr. 2023, Art. no. 109652.

[6] G. Wang and Y. Gu, "Multi-task scenario encrypted traffic classification and parameter analysis," *Sensors*, vol. 24, no. 10, p. 3078, May 2024.

[7] J.-T. Park, C.-Y. Shin, U.-J. Baek, and M.-S. Kim, "Fast and accurate multi-task learning for encrypted network traffic classification," *Appl. Sci.*, vol. 14, no. 7, p. 3073, Apr. 2024.

[8] A. Nascita, A. Montieri, G. Aceto, D. Ciuonzo, V. Persico, and A. Pescapé, "Improving performance, reliability, and feasibility in multimodal multitask traffic classification with XAI," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 1267–1289, Jun. 2023.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.

[9] F. Cerasuolo, I. Guarino, V. Spadari, G. Aceto, and A. Pescapé, "XAI for interpretable multimodal architectures with contextual input in mobile network traffic classification," in *Proc. IFIP Netw. Conf. (IFIP Networking)*, Jun. 2024, pp. 757–762.

[10] L. Garcia, G. Bartlett, S. Ravi, H. Ibrahim, W. Hardaker, and E. Kline, "Explaining deep learning models for per-packet encrypted network traffic classification," in *Proc. IEEE Int. Symp. Meas. Netw. (M&N)*, Jul. 2022, pp. 1–6.

[11] C. Molnar et al., "General pitfalls of model-agnostic interpretation methods for machine learning models," in *Proc. Int. Workshop Extending Explainable AI Beyond Deep Models Classifiers*. Cham, Switzerland: Springer, 2020, pp. 39–68.

[12] M. He, L. Jin, and M. Song, "Interpretability framework of network security traffic classification based on machine learning," in *Proc. Int. Conf. Artif. Intell. Secur.* Cham, Switzerland: Springer, 2021, pp. 305–320.

[13] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.

[14] Y. Liu, X. Wang, B. Qu, and F. Zhao, "ATVITSC: A novel encrypted traffic classification method based on deep learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 9374–9389, 2024.

[15] G. Xie, Q. Li, and Y. Jiang, "Self-attentive deep learning method for online traffic classification and its interpretability," *Comput. Netw.*, vol. 196, Sep. 2021, Art. no. 108267.

[16] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning (still) requires rethinking generalization," *Commun. ACM*, vol. 64, no. 3, pp. 107–115, Mar. 2021.

[17] L. D. Lingle, "Transformer-VQ: Linear-time transformers via vector quantization," in *Proc. 12th Int. Conf. Learn. Represent.*, 2023, pp. 1–30. [Online]. Available: https://openreview.net/forum?id=oDdzXQzP2F

[18] D. Bashir, G. D. Montañez, S. Sehra, P. S. Segura, and J. Lauw, "An information-theoretic perspective on overfitting and underfitting," in *Proc. 33rd Australas. Joint Conf. AI: Adv. Artif. Intell. (AI)*, Canberra, ACT, Australia. Cham, Switzerland: Springer, 2020, pp. 347–358.

[19] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, Feb. 2024, Art. no. 127063.

[20] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, 2016, pp. 407–414.

[21] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, I. Guyon et al.., Eds., Red Hook, NY, USA: Curran Associates, 2017, pp. 4765–4774. [Online]. Available: http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf

[22] P. Wang, X. Chen, F. Ye, and Z. Sun, "A survey of techniques for mobile service encrypted traffic classification using deep learning," *IEEE Access*, vol. 7, pp. 54024–54033, 2019.

[23] Z. Wang, "The applications of deep learning on traffic identification," *BlackHat USA*, vol. 24, no. 11, pp. 1–10, 2015.

[24] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, Jan. 2017, pp. 712–717.

[25] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Network traffic classifier with convolutional and recurrent neural networks for Internet of Things," *IEEE Access*, vol. 5, pp. 18042–18050, 2017.

[26] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapè, "MIMETIC: Mobile encrypted traffic classification using multimodal deep learning," *Comput. Netw.*, vol. 165, Dec. 2019, Art. no. 106944.

[27] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescape, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Trans. Netw. Service Manage.*, vol. 16, no. 2, pp. 445–458, Jun. 2019.

[28] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, New York, NY, USA, Apr. 2022, pp. 633–642, doi: 10.1145/3485447.3512217.

[29] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2023, vol. 37, no. 4, pp. 5420–5427. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/25674

[30] Z. Hang, Y. Lu, Y. Wang, and Y. Xie, "Flow-MAE: Leveraging masked AutoEncoder for accurate, efficient and robust malicious traffic classification," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2023, pp. 297–314.

[31] H. Huang, H. Deng, J. Chen, L. Han, and W. Wang, "Automatic multi-task learning system for abnormal network traffic detection," *Int. J. Emerg. Technol. Learn. (iJET)*, vol. 13, no. 4, p. 4, Mar. 2018.

[32] Y. Zhao, J. Chen, D. Wu, J. Teng, and S. Yu, "Multi-task network anomaly detection using federated learning," in *Proc. 10th Int. Symp. Inf. Commun. Technol.*, 2019, pp. 273–279.

[33] S. Rezaei and X. Liu, "Multitask learning for network traffic classification," in *Proc. 29th Int. Conf. Comput. Commun. Netw. (ICCCN)*, Aug. 2020, pp. 1–9.

[34] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, "DISTILLER: Encrypted traffic classification via multimodal multitask deep learning," *J. Netw. Comput. Appl.*, vols. 183–184, Jun. 2021, Art. no. 102985.

[35] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *Proc. Int. Conf. Mach. Learn.*, vol. 70, 2017, pp. 3319–3328.

[36] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 1135–1144.

[37] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim, "Sanity checks for saliency maps," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 9505–9515.

[38] P.-J. Kindermans et al., "The (Un)reliability of saliency methods," in *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Cham, Switzerland: Springer, 2019, pp. 267–280. [Online]. Available: https://doi-org.remotexs.ntu.edu.sg/10.1007/978-3-030-28954-6_14

[39] J. Early, G. K. C. Cheung, K. Cutajar, H. Xie, J. Kandola, and N. Twomey, "Inherently interpretable time series classification via multiple instance learning," 2023, *arXiv:2311.10049*.

[40] H. Turbé, M. Bjelogrlic, C. Lovis, and G. Mengaldo, "Evaluation of post-hoc interpretability methods in time-series classification," *Nature Mach. Intell.*, vol. 5, no. 3, pp. 250–260, Mar. 2023.

[41] H. D. Trinh, Á. Fernández Gambín, L. Giupponi, M. Rossi, and P. Dini, "Mobile traffic classification through physical control channel fingerprinting: A deep learning approach," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1946–1961, Jun. 2021.

[42] X. Chen et al., "TimeMIL: Advancing multivariate time series classification via a time-aware multiple instance learning," 2024, *arXiv:2405.03140*.

[43] Z. Chen et al., "Mod-squad: Designing mixtures of experts as modular multi-task learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 11828–11837.

[44] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2025, pp. 5998–6008.

[45] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.

[46] P. Dufter, M. Schmitt, and H. Schütze, "Position information in transformers: An overview," *Comput. Linguistics*, vol. 48, no. 3, pp. 733–763, Sep. 2022.

[47] U.-J. Baek, M.-S. Lee, J.-T. Park, J.-W. Choi, C.-Y. Shin, and M.-S. Kim, "Preprocessing and analysis of an open dataset in application traffic classification," in *Proc. 24th Asia–Pacific Netw. Oper. Manage. Symp. (APNOMS)*, Sep. 2023, pp. 227–230.

[48] J.-W. Son, S. Lee, and M.-H. Han, "Supervised service classification using downlink control indicator in LTE physical downlink control channel," in *Proc. Int. Conf. Inf. Commun. Technol. Converg. (ICTC)*, Oct. 2021, pp. 1533–1536.

[49] Y. Chen, Y. Tong, G. B. Hwee, Q. Cao, S. G. Razul, and Z. Lin, "Real-time traffic classification in encrypted wireless communication network," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2023, pp. 1–5.

[50] H. Ismail Fawaz et al., "InceptionTime: Finding AlexNet for time series classification," *Data Mining Knowl. Discovery*, vol. 34, no. 6, pp. 1936–1962, Nov. 2020.

[51] Z. Wang, W. Yan, and T. Oates, "Time series classification from scratch with deep neural networks: A strong baseline," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, May 2017, pp. 1578–1585.

[52] W. Samek, A. Binder, G. Montavon, S. Lapuschkin, and K.-R. Muller, "Evaluating the visualization of what a deep neural network has learned," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 11, pp. 2660–2673, Nov. 2017.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.



Yongming Chen (Graduate Student Member, IEEE) received the B.E. degree in communication engineering from Harbin Institute of Technology, Weihai, China, in 2020, and the M.S. degree in signal processing from Nanyang Technological University, Singapore, in 2022, where he is currently pursuing the Ph.D. degree with the School of Electrical and Electronic Engineering.

His research interests include machine learning, time-series analysis, and signal processing.



Hongsheng Lan received the B.S. degree in information engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2023, and the M.S. degree in signal processing from Nanyang Technological University, Singapore, in 2025.

His current research interests include traffic classification and traffic anomaly detection.



Bah Hwee Gwee (Senior Member, IEEE) received the B.Eng. degree from the University of Aberdeen in 1990 and the M.Eng. and Ph.D. degrees from Nanyang Technological University in 1992 and 1998, respectively.

He is currently an Associate Professor and an Associate Chair (Students) with the School of EEE, NTU, and the Director of the National Integrated Centre for Evaluation (NiCE). He has led research projects totaling over USD 15 million, including major grants from MOE Tier-2, ASEAN-EU, DSO, NTU-Panasonic, US DARPA, A*STAR, CSA, and NRF. He has published more than 200 articles, five U.S. patents granted, and founded two start-ups. His research interests include machine learning, hardware security, image processing, and asynchronous circuits.



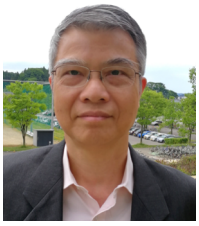
Qi Cao received the B.Eng. degree from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2000, and the Ph.D. degree from Nanyang Technological University, Singapore, in 2007.

Since 2024, he has been an Associate Professor with the School of Computing Science, University of Glasgow, U.K. From 2019 to 2024, he was an Assistant Professor with the School of Computing Science, University of Glasgow. He will join the Department of Statistics and Data Science, National University of Singapore, as an Associate Professor (Practice), in January 2026. His research interests include computational intelligence, data analytics, software engineering education, and technology enhanced learning with virtual reality (VR) and augmented reality (AR).



Sirajudeen Gulam Razul received the B.Eng. and M.Eng. degrees in electrical and electronics engineering from Nanyang Technological University, Singapore, in 1997 and 2000, respectively, and the Ph.D. degree from the University of Cambridge, U.K., in 2003. His Ph.D. work was on Bayesian estimation using Markov Chain Monte Carlo techniques.

From 2004 to 2009, he was a Faculty Member with the School of EEE, Nanyang Technological University. He joined Temasek Laboratories@NTU in 2009. He is currently the Program Director of the Centre for Communications and Signal Processing, TL@NTU. His research interests include Bayesian signal processing, statistical signal processing, and array processing.



Zhiping Lin (Life Senior Member, IEEE) received the Ph.D. degree in information engineering from the University of Cambridge, U.K., in 1987.

Since 1999, he has been an Associate Professor with the School of EEE, Nanyang Technological University (NTU), Singapore. He is currently the Program Director of the Master of Science, School of EEE, NTU. Prior to that, he was with DSO National Laboratories, Singapore, and Shantou University, China. He has published more than 400 journal articles and conference papers. His research interests include signal processing and machine learning. He was the Editor-in-Chief of *Multidimensional Systems and Signal Processing* from 2011 to 2015 and served as an associate editor for several other international journals. He was an IEEE CASS Distinguished Lecturer and served as IEEE CASS and SPS Singapore Chapter Chairs.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:15:31 UTC from IEEE Xplore. Restrictions apply.